



Module 2. Supervised Learning and Experimental Design

Decision Trees - Intro

Creating a Decision Tree

Splitting Models

Visualizing a Decision Tree

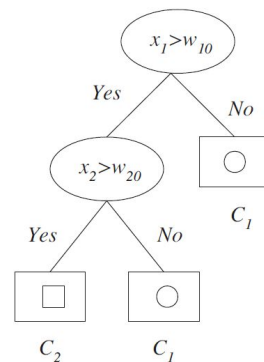
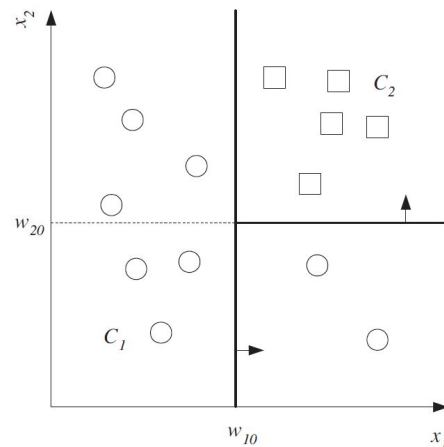
Decision Trees and Overfitting

Introduction

- A decision tree is a **non-parametric discriminative** classifier. **That can be used for classification and regression!**
- In particular, a decision tree is a hierarchical model for supervised learning whereby the local region (in the feature space) is identified in a sequence of recursive splits in a smaller number of steps.
- There are many models to measure the quality of the split.
- The splitting models for classification tasks look at maximize information gain or minimize impurity.
- The learning algorithm is also known as CART which stands for Classification and Regression Tree.

Introduction

- A decision tree is composed of internal decision nodes and terminal leaves.
- Each decision node m implements a test function $f_m(x)$ with discrete outcomes labeling the branches.
- Given an input, at each node, a test is applied and one of the branches is taken depending on the outcome.
- This process starts at the root and is repeated recursively until a leaf node is hit, at which point the value written in the leaf constitutes the output.



Introduction

- A decision tree is a nonparametric model in the sense that we do not assume any parametric form for the class densities
 - We don't have a mapper function. **So, how are we going to learn the weights!** → based on the metric we decide to use
- The tree structure is not fixed a priori. The tree grows. Branches and leaves are added, during learning depending on the complexity of the problem inherent in the data.

Introduction

- Each leaf node has an output label, which in the case of classification is the class code and in regression is a numeric value.
- A leaf node defines a localized region in the input space where instances falling in this region have the same labels (in classification) or very similar numeric outputs (in regression).

Advantages

- The boundaries of the regions are defined by the discriminants that are coded in the internal nodes on the path from the root to the leaf node.
- The hierarchical placement of decisions allows a fast localization of the region covering an input.
- For example, if the decisions are binary, then in the best case, each decision eliminates half of the cases.
- Another advantage of the decision tree is interpretability. As we will see shortly, the tree can be converted to a set of IF-THEN rules that are easily understandable.



Creating Decision Trees

Creating a Decision Tree

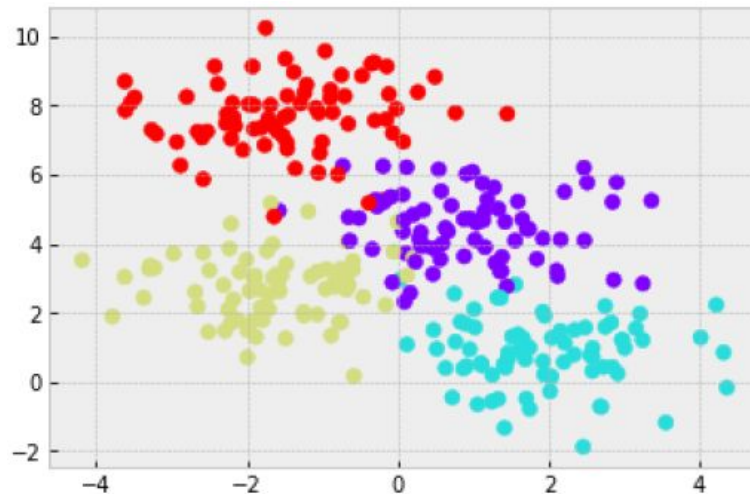
- Consider the following two-dimensional data, which has one of four class labels:

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
plt.style.use('bmh')

from sklearn.datasets import make_blobs

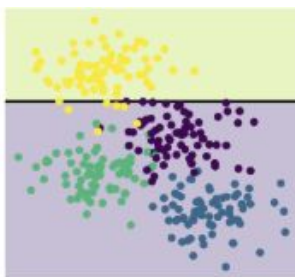
X, t = make_blobs(n_samples=300, centers=4,
                  random_state=0, cluster_std=1.0)

plt.scatter(X[:, 0], X[:, 1], c=t, s=50, cmap='rainbow');
```

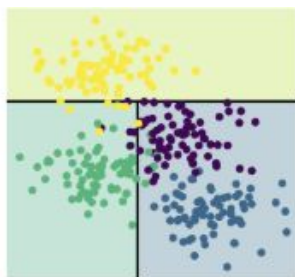
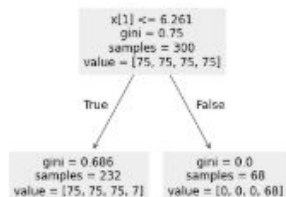


Creating a Decision Tree

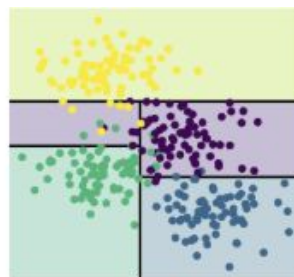
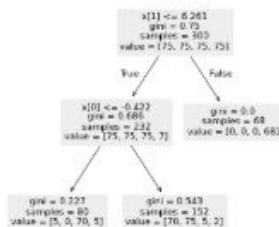
A simple decision tree built on this data will iteratively split the data along one or the other axis according to some quantitative criterion, and at each level assign the label of the new region according to a majority vote of points within it. This figure presents a visualization of the first four levels of a decision tree classifier for this data:



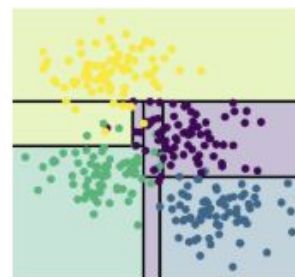
depth = 1



depth = 2



depth = 3



depth = 4

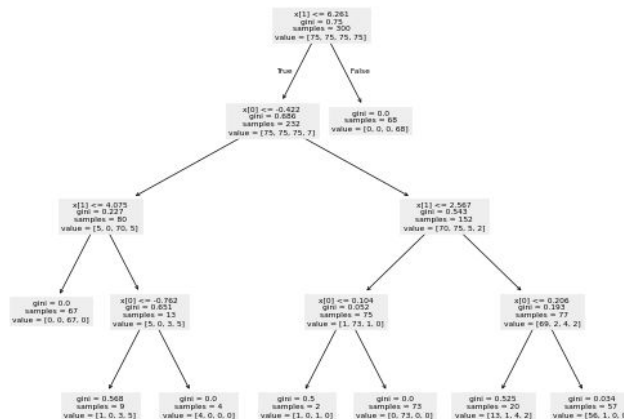


Creating a Decision Tree

- Notice that after the first split, every point in the upper branch remains unchanged, so there is no need to further subdivide this branch.
- Except for nodes that contain all of one color, at each level every region is again split along one of the two features. This process of fitting a decision tree to our data can be done in Scikit-Learn with the `DecisionTreeClassifier` estimator.

```
dt = DecisionTreeClassifier(max_depth=4, random_state=0)
dt.fit(X,t)

tree.plot_tree(dt);
```

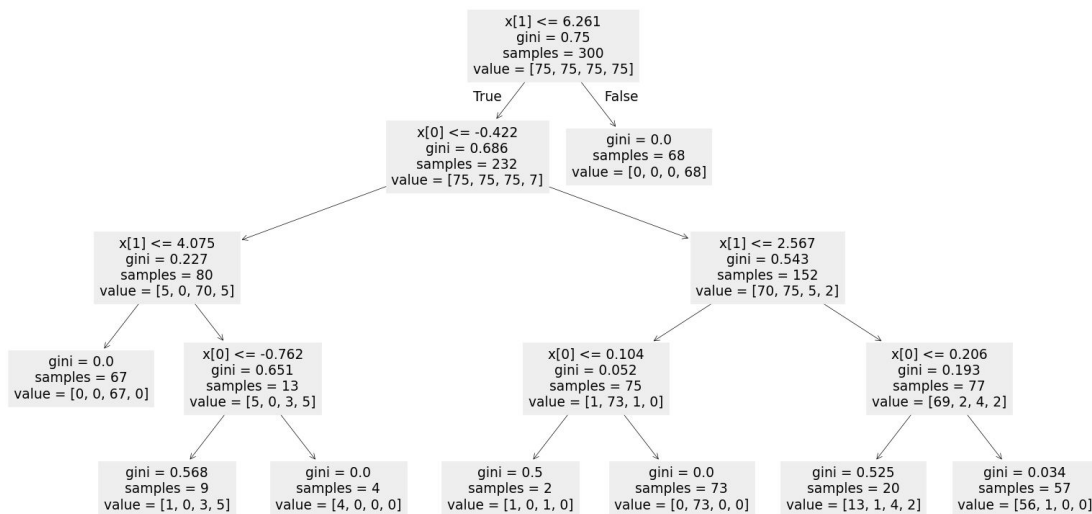




Splitting Models

Impurity and Information Criteria

- The **impurity** in the name suggests that the aim of the decision tree is to have each leaf node represent a set of data points that are in the same class, so that there are no mismatches. This is known as **purity**.
- If a leaf is **pure** then all of the training data within it have just one class.



Impurity and Information Criteria

There are many models to measure the quality of the split. The most common impurity criteria are:

- **Entropy**: is an information gain split criterion. Measure of uncertainty or disorder in a dataset.

$$I_H = - \sum_{j=1}^c p_j \log_2(p_j)$$

- **Gini Index**: is an impurity split criterion.

$$I_G = 1 - \sum_{j=1}^c p_j^2$$

Where p_i is the proportion of samples that belong to class i for a particular node

Impurity and Information Criteria

Each split uses the feature that maximizes the information gain:

The weighted
sum of individual
impurity
measures

$$IG(D_p, f) = I(D_p) - \frac{N_{\text{left}}}{N_p} I(D_{\text{left}}) - \frac{N_{\text{right}}}{N_p} I(D_{\text{right}})$$

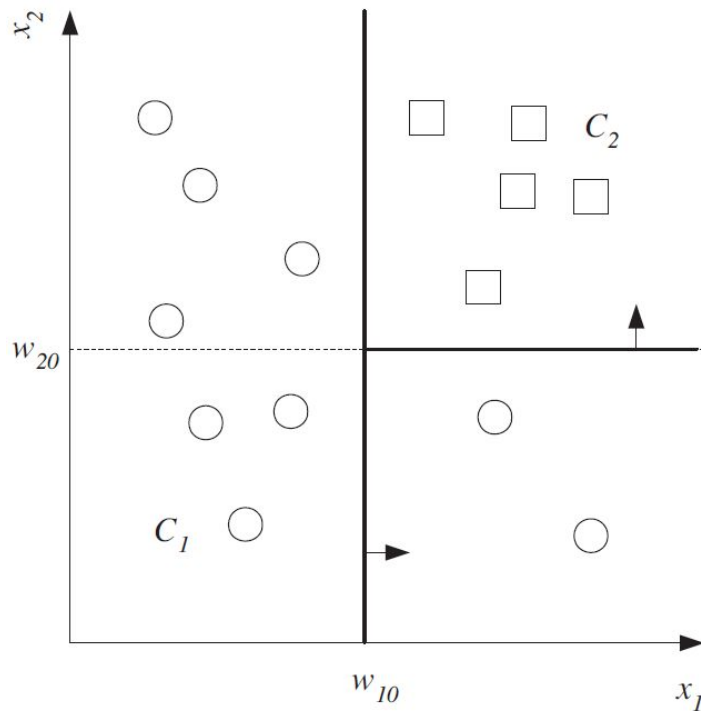
The information gain is going to be the objective function

We will work on the Exercise
included in the Jupyter
Notebook next class!!

Impurity and Information Criteria

The goal is to find the feature to make the partition and extract the data

X_1 has the most information gain because when we split it at w_{10} anything on the left is classified as class 2





Visualizing a Decision Tree

Plotting the tree

```
from sklearn.tree import plot_tree

dt = DecisionTreeClassifier(max_depth=4, random_state=0)
dt.fit(X, t)

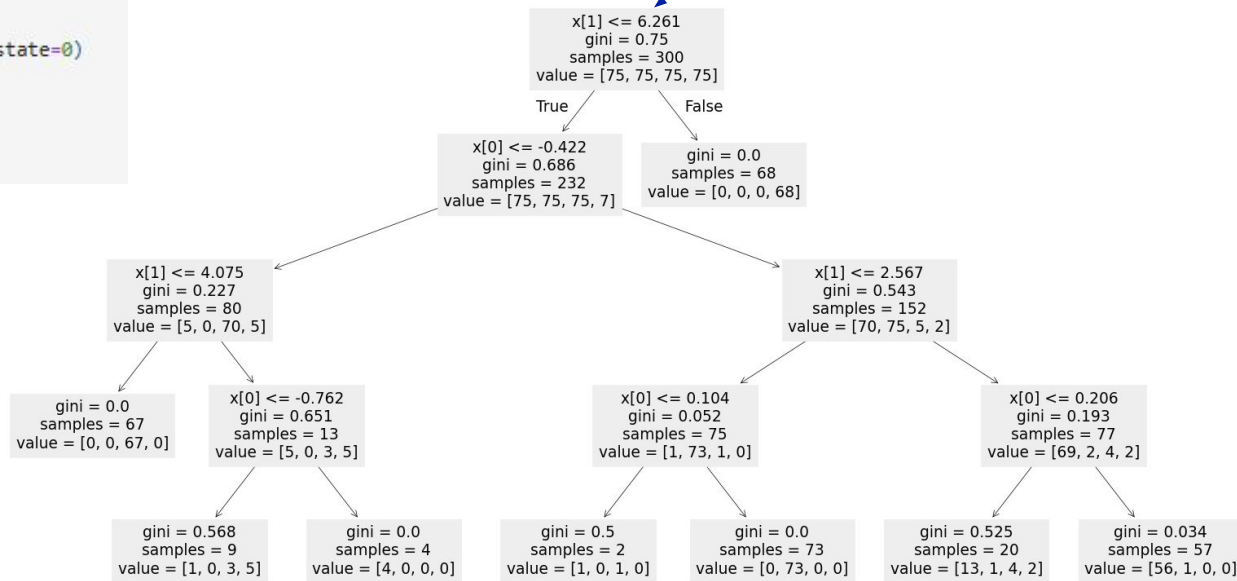
plt.figure(figsize=(20, 10))
plot_tree(dt);
```

← True

→ False

Depth: 4

Feature used to make the cut



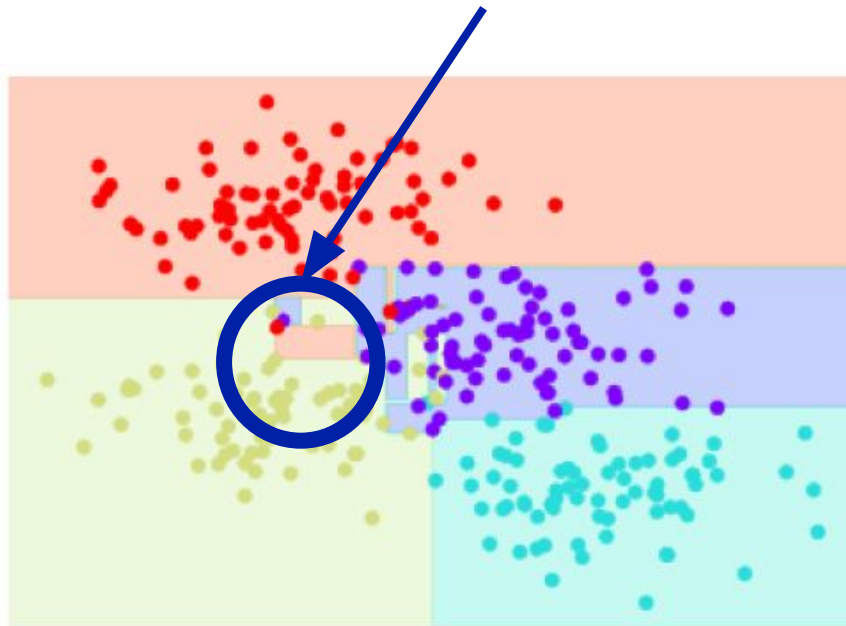
Only 2
samples!

Pure
leaf

Use the function in the Notebook!

Using the function in the corresponding Jupyter Notebook will allow you to illustrate how this decision surface changes as you increase the tree depth

What can you see here?



A series of white lines on the left side of the slide, consisting of several horizontal lines that are slightly offset from each other, creating a sense of depth and movement.

Estimating Class Probabilities

Estimating Class Probabilities

- A Decision Tree can also estimate the probability that an instance belongs to a particular class. First it traverses the tree to find the leaf node for this instance, and then it returns the ratio of training instances of class in this node

```

X[:2]       $X_0$        $X_1$ 

array([[ 0.02458305,  2.96215652],
       [-1.52392631,  7.12573205]])

dt.predict_proba(X[:2])

array([[0.8961039, 0.02597403, 0.05194805, 0.02597403],
       [0., 0., 0., 1.]])
    
```

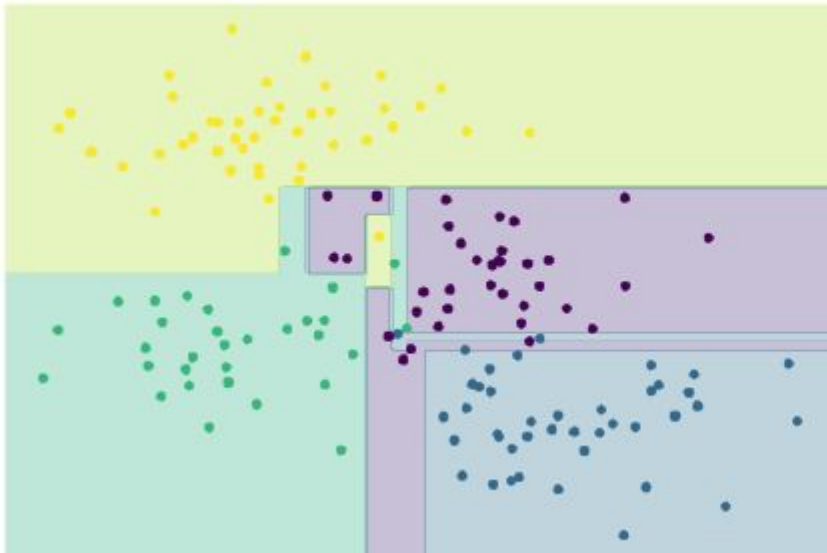
- Two samples and two features



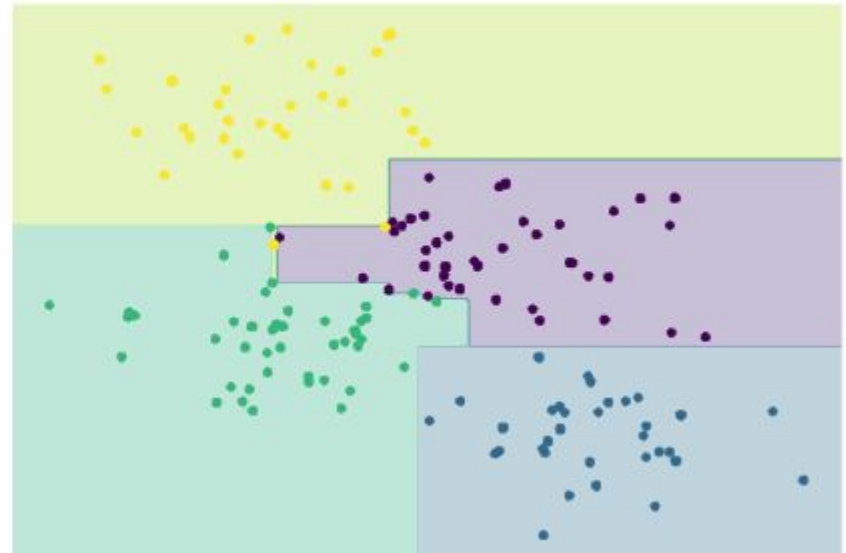
Decision Trees and Overfitting

Overfitting

- Overfitting turns out to be a general property of decision trees: it is very easy to go too deep in the tree, and thus to fit details of the particular data rather than the overall properties of the distributions they are drawn from.



Training



Test

Regularization Hyperparameters

- Decision trees make very **few assumptions** about the training data (as opposed to linear models, which assume that the data is linear, for example).
- If left unconstrained, the tree structure will adapt itself to the training data, fitting it very closely - indeed, most likely overfitting it.
- Such a model is often called a **nonparametric model**, not because it does not have any parameters (it often has a lot) but because **the number of parameters is not determined prior to training**, so the model structure is free to stick closely to the data.
- In contrast, a parametric model, such as a linear model, has a predetermined number of parameters, **so its degree of freedom is limited**, reducing the risk of overfitting (but increasing the risk of underfitting).



Regularization Hyperparameters

We need to add regularization!

- The regularization hyperparameters depend on the algorithm used, but generally you can at least restrict the maximum depth of the decision tree.
- In Scikit-Learn, this is controlled by the `max_depth` hyperparameter (the default value is `None`, which means unlimited).
- Reducing `max_depth` will regularize the model and thus reduce the risk of overfitting.

Regularization Hyperparameters

- The DecisionTreeClassifier class has a few other parameters that similarly restrict the shape of the decision tree:
 - **min_samples_split** (the minimum number of samples a node must have before it can be split),
 - **min_samples_leaf** (the minimum number of samples a leaf node must have),
 - **min_weight_fraction_leaf** (same as min_samples_leaf but expressed as a fraction of the total number of weighted instances),
 - **max_leaf_nodes** (the maximum number of leaf nodes), and
 - **max_features** (the maximum number of features that are evaluated for splitting at each node). Increasing min_* hyperparameters or reducing max_* hyperparameters will regularize the model.

Using Cross-Validation!!

Pruning the Tree

- Other algorithms work by first training the decision tree without restrictions, then pruning (deleting) unnecessary nodes.
- A node whose children are all leaf nodes is considered unnecessary if the purity improvement it provides is not statistically significant
 - **Scikit Learn does not use this! → Pre-pruning not post-pruning**

